Deep Reinforcement Learning Project Report

Haoxu Huang **Zeyang Zhang** **Chengwei Liang**

Abstract

Real-world Reinforcement Learning (RL) encounters great difficulty in robotic tasks, stemming from the complexity and high cost of physical interaction. To handle this problem, prior knowledge such as human demonstrations can be leveraged to improve sample efficiency of real-world RL. In this project, we train a robot to shoot a basketball using real-world RL. We adopt two methods to incorporate prior knowledge: 1) initialize the policy network with behavior cloning from demonstration data, and 2) integrate offline data into online training using RLPD. We first validate our approach in simulation, where we show that both methods significantly improve sample efficiency and policy performance. We then apply our methods to the real-world basketball shooting task. However, we encounter critical safety issues during real-world training, which prevent us from completing the training. We analyze the causes of these issues and discuss potential solutions. Our findings highlight the importance of offline data fidelity, safety mechanisms, and careful exploration-exploitation balance in real-world RL. Our code and supplementary video are available at `https://github.com/HaoxuHuang/serl`.

1 Introduction

This project tackles the challenging task of robotic basketball shooting using a Franka robot arm. Successfully throwing a ball into a target basket requires precise control of complex physical dynamics—including release timing, velocity profiles, and trajectory prediction—under real-world constraints. While reinforcement learning offers a promising framework for learning such dynamic skills, its practical application faces significant hurdles.

Our methodological approach is motivated by two key challenges in robotic RL. First, simulation-based training presents fundamental limitations: developing high-fidelity basketball physics requires substantial engineering effort; imperfect dynamics models create sim-to-real transfer gaps; and unmodeled environmental noises degrade policy performance. We therefore bypass simulation entirely and focus on learning directly in the real world. Second, real-world training demands exceptional sample efficiency due to practical constraints. This necessitates RL approaches that maximize learning progress from limited physical interactions.

To address these requirements, we investigate sample-efficient real-world RL methods that leverage prior demonstration data. Our implementation builds on the SERL framework for asynchronous real-world training, combined with:

1. Behavior Cloning (BC) for policy initialization
2. RLPD [1] that integrates offline data during online learning
3. Hybrid approaches that combine BC and RLPD to exploit demonstration data

We experiment with these methods in both simulation and real-world settings. In simulation, we validate the correctness and sample efficiency of our approach. Subsequently, we apply our methods to the real-world basketball shooting task to evaluate their effectiveness under realistic physical dynamics and hardware constraints.

Preprint. Under review.

The reminder of this report is organized as follows: Section 2 reviews related work on efficient RL, real-world RL frameworks, and basketball shooting tasks. Section 3 describes our approach to the basketball shooting task, including problem formulation and methods. Section 4 presents the experimental setup and results in both simulation and real-world settings. Finally, we conclude in Section 5.

2 Related Work

Efficient Reinforcement Learning. Efficiency is crucial in reinforcement learning, particularly for real-world applications. Model-based methods such as Dreamer [3], MuZero [8], and MOPO [11] learn environment dynamics from data and generate imagined samples to improve sample efficiency. Offline RL algorithms like RLPD [1], CQL [6], and IQL [5] leverage static datasets to provide useful priors for policy learning. In this work, we adopt the RLPD algorithm, which incorporates demonstration data into the Soft Actor-Critic (SAC) [2] framework.

Real-World RL Frameworks. Several mature frameworks facilitate real-world reinforcement learning. DayDreamer [10] extends Dreamer [3] to robotic applications by providing infrastructure for world-model-based training on hardware. ARIEL [9] integrates offline data from diverse tasks to enable sample-efficient, zero-shot, and reset-free policy learning on physical robots. SERL [7] offers a modular, asynchronous RL suite where actor nodes collect experience and learner nodes update policy and value networks from a shared replay buffer. Our implementation builds on SERL to support scalable, real-time training on a robotic platform.

Basketball Shooting Task. ADAP [4] proposes an agile control algorithm for real-world tasks and demonstrates its effectiveness on basketball-shooting task. It extracts open-loop motion primitives from high-quality demonstrations and refines them through a limited number of real-world trials. However, ADAP's performance depends heavily on demonstration quality. By contrast, our method integrates reinforcement learning to exploit suboptimal offline data and continually improve policy performance through online interaction.

3 Approach

In this section, we describe our approach to the basketball shooting task using reinforcement learning. We first formulate the problem as a reinforcement learning task, then describe the SERL framework we use for asynchronous reinforcement learning, and finally introduce the RL agents we implement and compare.

3.1 Problem Formulation

Our goal is to throw a ball to a specified target on the ground using a Franka robot arm. We cast this task as a reinforcement learning problem by defining its state space, action space, transition dynamics, and reward function as follows:

- **State Space.** We include only the robot's proprioceptive information—its joint angles and velocities. Since the robot base and target are fixed, and the ball always starts at a known grasp point on the end effector and does not recontact the arm after release, we omit the ball's position. The robot's joint configuration fully determines the launch conditions relative to the target.
- **Action Space.** Actions correspond to desired joint-velocity commands. At each control step (approximately 50 Hz), the robot applies these joint velocities for a short duration, causing the arm to move and the ball to be launched.
- **Transition Dynamics.** The environment evolves according to the robot's and ball's physical dynamics. An episode terminates when the ball contacts the ground, the robot reaches its safety threshold, or a maximum time horizon is exceeded.
- **Reward Function.** We employed a γ-discount reward scheme. Once the ball hits the ground, we assign a distance-based reward

$$r_D = \lambda_1\big(u - \mathrm{dist}(\text{ball, target})\big),$$

where u is a baseline distance and dist measures the landing distance.

To encourage smooth motions and reduce drift from the start pose s_0, we impose two penalties:

$$r_{\text{energy}} = -\lambda_2 \|a\|_2, \quad r_{\text{pos}} = -\lambda_3 \|s - s_0\|_2.$$

Additionally, to enforce safety constraints, we introduce a constant safety penalty r_{safety} that activates when the robot approaches its predefined safety threshold,

The total reward is then

$$r = r_D + r_{\text{energy}} + r_{\text{pos}} + r_{\text{safety}}.$$

Here, $\lambda_1, \lambda_2, \lambda_3, r_{\text{safety}}$ are hyperparameters that balance accuracy, energy efficiency, pose consistency, and safety compliance.

3.2 Methods

We develop an asynchronous reinforcement learning pipeline over the SERL framework [7]. In SERL, two types of processes run concurrently:

- **Actor Node.** The actor interacts with the physical environment at real-time control rate, executing the current policy to collect transitions (s, a, r, s'). After each episode or fixed number of steps, it sends these samples to a replay buffer and periodically pulls the latest policy weights from the learner.
- **Learner Node.** The learner continuously samples mini-batches from the shared buffer to update the policy and value networks using off-policy reinforcement learning algorithms. Every few gradient-update steps, it broadcasts the updated network parameters back to the actors, ensuring they execute a near-up-to-date policy.

The overall architecture of SERL is illustrated in Figure 1.

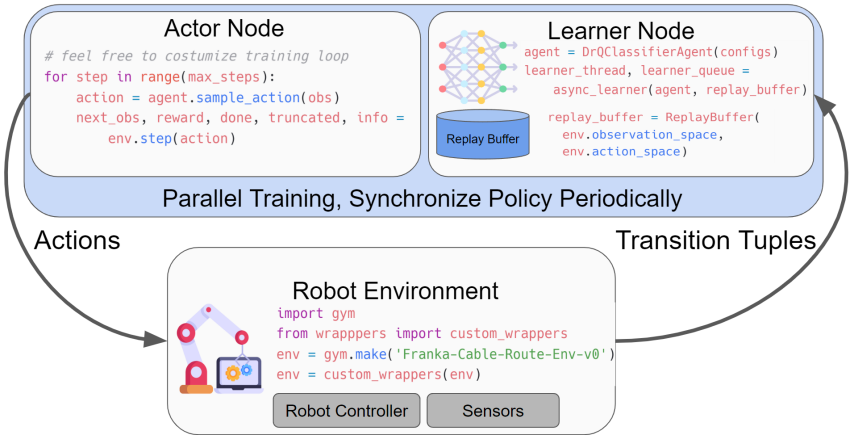


Figure 1: SERL framework for asynchronous reinforcement learning. The actor nodes interact with the environment and send data to a shared replay buffer, while the learner node updates the policy and value networks using off-policy RL algorithms. Picture is adopted from [7].

Within this framework, we implement and compare four RL agents:

1. **Baseline SAC.** Standard off-policy SAC with no additional data sources: the learner draws all training batches from the replay buffer accumulated by the actors.
2. **SAC with BC.** Pretrain a policy network on demonstration trajectories via behavior cloning, minimizing $\mathcal{L}_{\text{BC}} = \mathbb{E}_{(s,a)\sim\mathcal{D}_{\text{demo}}}\big[-\log \pi_\phi(a|s)\big]$. Initialize the SAC actor network with the BC-pretrained weights before starting the asynchronous training.
3. **SAC with RLPD.** During online training, form each mini-batch by sampling a fraction $p\%$ from the demonstration dataset and $(100-p)\%$ from the live replay buffer; the mixing ratio p decays over time, shifting the emphasis from offline to online data.

4. **SAC with BC and RLPD.** This hybrid approach combines both behavior cloning initialization and the offline-online mixed-batch training schedule described above, aiming to leverage demonstrations for faster convergence and stable policy improvement.

4 Experiments

In this section, we describe the experimental setup and report the results for the basketball shooting task. We begin by conducting experiments in simulation to verify the correctness and sample efficiency of our approach. Following successful validation, we deploy our methods in the real world to evaluate their performance under realistic physical dynamics and hardware constraints.

4.1 Experiments in Simulation

We conducted extensive simulation experiments to validate our approach. In our MuJoCo setup, the Franka arm is mounted at the origin $(0, 0)$ on the ground plane, with the landing target fixed at $(1.50,\ 0.25)$ on the ground. To approximate real-world delays from manual resets, we insert a 7-second pause between episodes. Control commands are sended at 50 Hz, while the simulated physics engine runs at 500 Hz to ensure high-fidelity dynamics.

To generate demonstration data for BC and RLPD, we first trained a standard SAC agent in simulation without the inter-episode pause or any real-time constraints. To introduce a consistent bias in these demos, in this experiment we placed the training target at $(1.50,\ 0.00)$ rather than $(1.50,\ 0.25)$. After approximately 2.5 hours of training (~1M actor steps and ~500K learner updates), the agent achieved a mean landing error of 0.2 m. We then collected 3000 transitions from this policy, which served as our offline dataset. Some demonstration trajectories are shown in Figure 3.

We performed a comprehensive hyperparameter study to identify settings that yield stable and efficient learning, since our experiments revealed a strong sensitivity to these parameters. In preliminary trials, we observed that relatively small values of the distance-reward coefficient λ_1 caused the agent to remain stationary rather than attempt throws. Additionally, reducing the discount factor γ below 0.99 caused the agent to release the ball prematurely, effectively "dropping" it instead of executing the intended throwing motion. To balance accuracy and exploration, we fixed $\lambda_1 = 10$, $\lambda_2 = 10^{-4}$, $\lambda_3 = 10^{-5}$ and disabled the safety penalty. Finally, we set the discount factor to $\gamma = 0.999$ to emphasize long-horizon returns, which proved critical for high-speed, fine-grained throws.

Using these settings, we trained each of the four agents described in Section 3.2 twice in simulation. The training is conducted for approximately 2.5 hours per run, with 500K learner steps and 60K actor steps. Figure 2 plots the resulting learning curves.

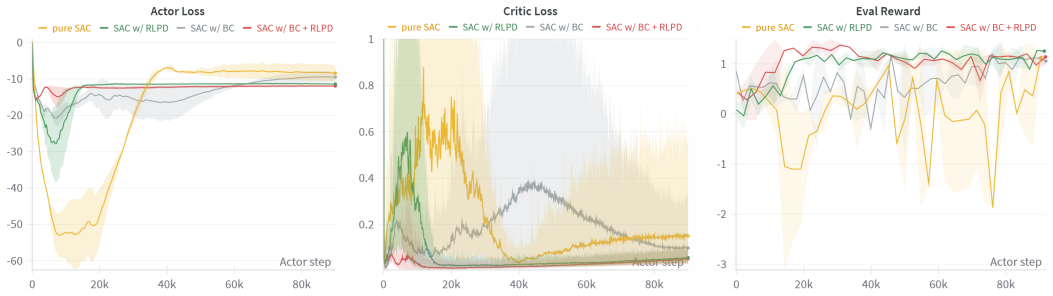


Figure 2: Training curve for the four RL agents in simulation. The shaded area represents the standard deviation over two runs.

We observe that the baseline SAC agent failed to converge to a stable throwing policy within the allotted steps. We attribute this to the long-horizon, delayed-reward nature of the task, which demands learning a complex motion before receiving meaningful feedback. Its poor sample efficiency also resulted in high variance, hindering reliable convergence.

Introducing demonstration priors—either via BC or RLPD—substantially improved both efficiency and stability. Among the three agents leveraging offline data, SAC+BC underperformed relative to SAC with RLPD alone or the hybrid approach. We hypothesize this is because SAC+BC only exploits demos at initialization, whereas the other methods continuously draw on the offline dataset throughout training. The combined SAC+BC+RLPD agent achieved the fastest convergence and lowest variance, demonstrating that initializing from demos and maintaining offline guidance synergize to accelerate and stabilize training. Moreover, all agents with priors surpassed the quality of the original demonstration trajectories, confirming that RL can refine suboptimal offline data. Figure 4 illustrates example trajectories from the SAC+BC+RLPD agent.

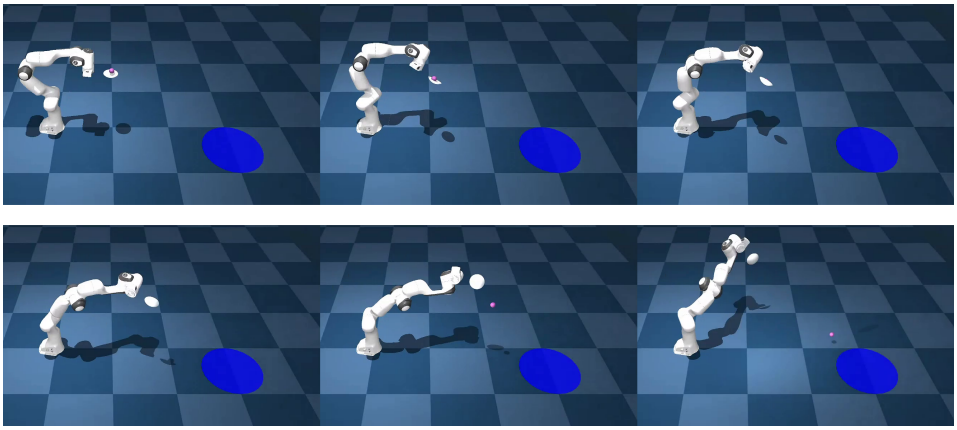


Figure 3: Demonstration trajectories collected from the SAC agent in simulation.

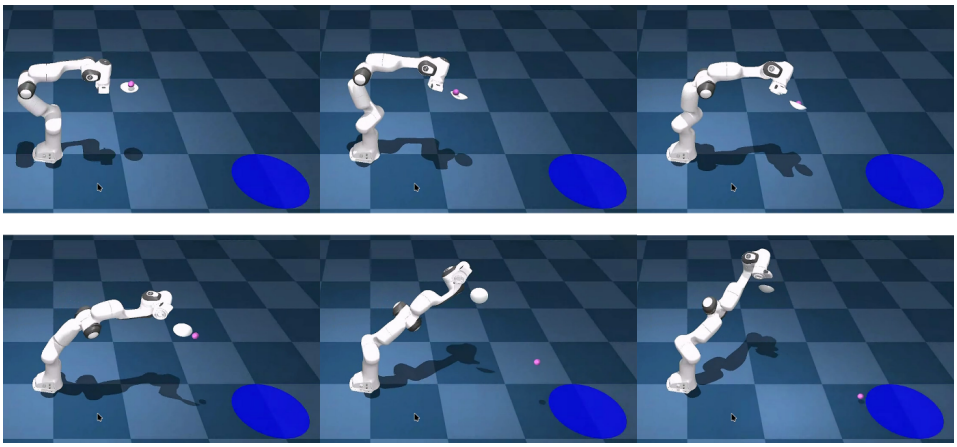


Figure 4: Trajectories of the SAC with BC and RLPD agent in simulation. The center of the blue circle on the ground is the target position.

4.2 Experiments in the Real World

We conducted experiments in the real world to evaluate the performance of our RL agents under physical dynamics and hardware constraints. However, we encountered significant safety issues during training, which we detail and analyze in the following sections.

4.2.1 Preparation

After validating our approach in simulation, we deployed the agents on a real Franka robot arm. The robot was mounted with its base approximately 0.8 m above the ground, and a fixed target was placed on the ground about 1 m in front of the arm. A camera system automatically detects when the ball contacts the ground and measures its landing position to compute rewards.

We first collected demonstration trajectories by manually guiding the robot arm through the shooting motion, as shown in Figure 8. In total, we recorded 30 trajectories (~3000 steps). Since when collecting demonstrations, the robot arm was controlled manually, not by the robot's motor commands, we could not directly obtain the action labels for these demonstrations. Hence, we estimated the actions by finite differences of successive observations.

We used this dataset to pretrain a policy via BC. However, when rolling out the cloned policy on the robot, we observed that it produced erratic motions. We attribute this failure to the imprecise action estimates in the demonstration data. As shown in Figure 9, the estimated actions is a rough approximation that captures direction but not precise magnitude. Policy cloning on this noisy data leads the agent to out-of-distribution states and irrational behaviors.

To mitigate overfitting to the noisy action labels and improve generalization, we added an ℓ_2 regularization term to the BC loss. This yields a policy that reliably moves the end effector toward the target, although it lacks the upward velocity profile required for a complete throw.

To further enhance the offline data quality, we rolled out the regularized BC policy for 100 trajectories (~20000 steps) on the real robot. These rollouts produce consistent state-action-next-state tuples under the true dynamics and camera-based reward feedback. We then combined these new transitions with the original manual demonstrations to form a richer offline dataset for subsequent training.

4.2.2 SAC with BC and RLPD

Our first attempt to train the RL agents in the real world employed SAC with BC and RLPD. However, we found the learned policy to be unstable during the initial training phase. Specifically, within the first 200 learner steps, the action magnitude increased rapidly, causing the robot to move approximately 3x faster than the BC policy. Figure 5 illustrates this behavior.

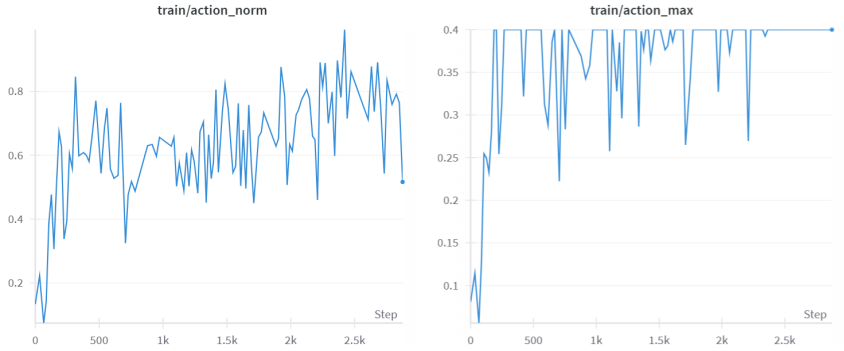


Figure 5: Action magnitude when training SAC with BC. The action is a 7-dimensional vector, each component representing a joint velocity of the robot arm. `action_norm` denotes the L2 norm of the action vector, and `action_max` denotes its maximum absolute component. During the first 200 learner steps, `action_norm` rises from 0.2 to 0.6, and `action_max` rises from 0.1 to 0.4, reaching our predefined action limit.

This instability poses no major issue in simulation, where random exploration is tolerated. In the real world, however, the high action magnitudes raise serious safety concerns. We cannot further constrain these magnitudes, since tasks like ball throwing require sufficient joint velocities. Even after carefully designing safety bounds on joint angles and end-effector positions, the robot frequently hits its safety limits, triggering emergency stops. Consequently, we were unable to complete real-world training of SAC with BC and RLPD.

Upon analysis, we attribute the excessive action magnitudes to a high temperature in SAC, which encourages high-entropy policies and thus increased exploration. As shown in Figure 6, the standard deviation of the action distribution increases rapidly during the first 300 learner steps. Although the norm of the distribution's mode remains moderate, sampling produces actions too large for safe execution. After approximately 1800 learner steps, even the mode begins to diverge, likely because the inflated variance hinders the critic from learning an reasonable value function. This, in turn, leads to random policy updates and eventual divergence.

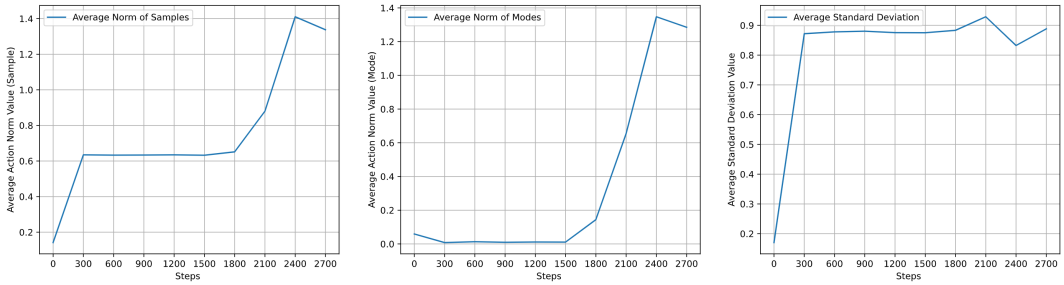


Figure 6: Standard deviation of the action distribution under SAC with BC. The rapid increase in standard deviation during the first 300 learner steps drives high exploration and large action magnitudes. After around 1800 steps, the mode of the distribution also diverges, likely as a consequence of this high variance.

Figure 11 shows representative trajectories of the SAC-with-BC agent in the real world at 0, 900, 1800, and 2700 learner steps, highlighting the policy's divergence over time.

4.2.3 SAC with Offline RL

We also attempted to train the RL agents using SAC with offline RL as prior. The offline RL is performed using the union of the demonstration data and the replay buffer of the BC policy. However, we encountered similar issues as with SAC with BC alone. Both the magnitude of the mode and the samples from the action distribution increased to unsafe levels during the offline RL phase, as shown in Figure 7. This led to the same safety issues as before, preventing us from beginning the online RL.

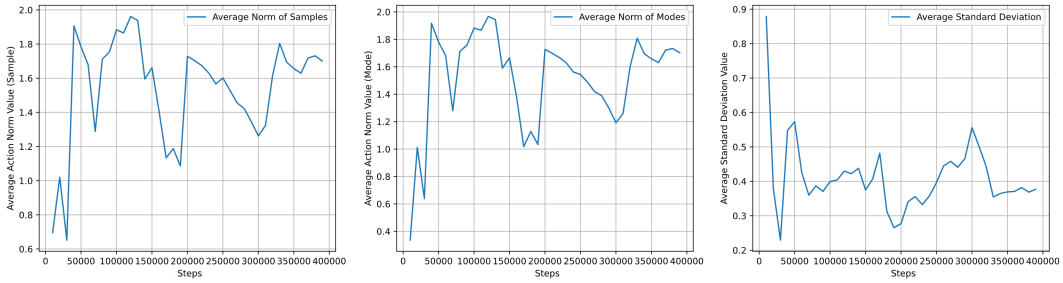


Figure 7: Magnitude and standard deviation of the action distribution under evaluation of SAC with BC and offline RL. The action magnitude increases to unsafe levels in the early stage and remains high later, even after the standard deviation drops to a low range.

The trajectories of the offline RL result are shown in Figure 10.

We attribute this issue to (1) the limited fidelity of the demonstration data, and (2) the absence of negative samples in the offline data. As shown in Section 4.2.1, the collected demonstration data is not fully aligned with the real-world robot dynamics, which leads to a biased policy. The replay buffer of the BC policy contains only positive samples, i.e., actions that lead to successful shots. Specifically, trajectories with high action magnitudes are poorly represented in the offline data, since the BC policy is trained to imitate the demonstration data, which does not contain such actions. As a result, the offline RL algorithm learns a biased value function that overestimates the value of high action magnitudes, leading to unsafe actions during training.



Figure 8: Manual demonstration data collection in the real world.

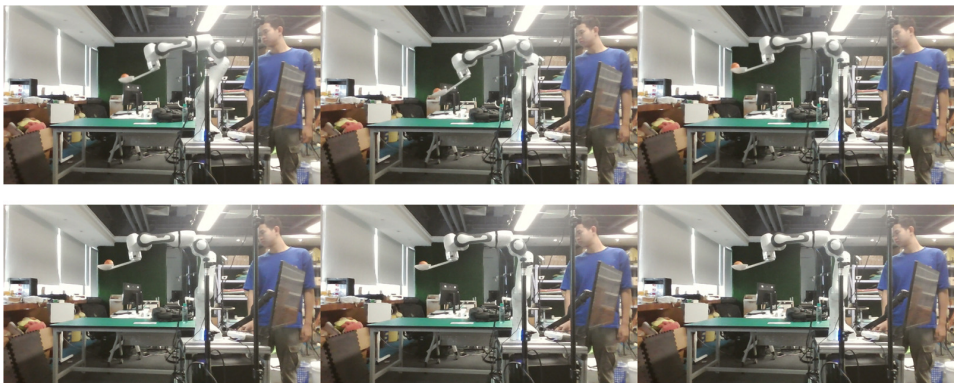


Figure 9: Replaying estimated demonstration actions on the robot: the motion pattern is similar to the manual demonstrations, but the arm fails to launch the ball.



Figure 10: Trajectories of the offline RL agent in the real world.

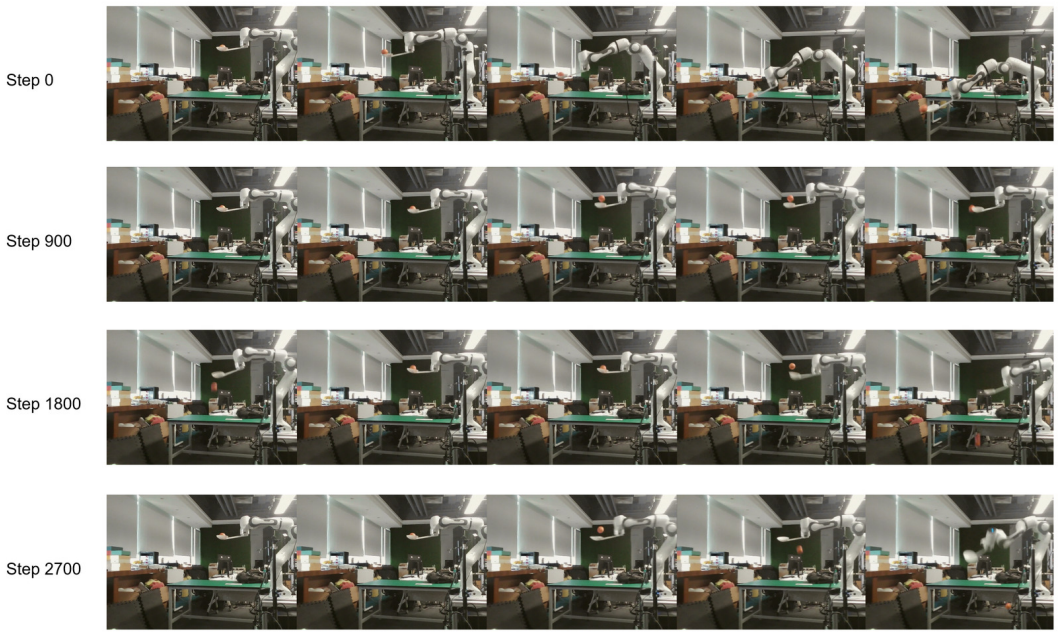


Figure 11: Trajectories of the SAC-with-BC agent in the real world at various training stages (0, 900, 1800, and 2700 learner steps).

5 Conclusion

In this project, we investigated sample-efficient real-world reinforcement learning methods for robotic basketball shooting using a Franka arm. Our approach leveraged prior demonstration data through two complementary techniques: BC for policy initialization and RLPD for offline-to-online knowledge transfer. Simulation experiments demonstrated that both methods significantly improved sample efficiency compared to standard SAC, with the combined approach achieving the fastest convergence and lowest variance. Notably, all prior-based methods surpassed the performance of the original demonstrations, confirming RL's ability to refine suboptimal offline data.

However, real-world deployment revealed critical challenges. Manual demonstrations suffered from imprecise action labels due to finite-difference estimation, leading to ineffective BC policies. During online training, we observed dangerous action inflation in both SAC+BC+RLPD and SAC+offline RL approaches, causing unsafe behaviors that triggered safety stops. This divergence stemmed from: 1) limited-fidelity demonstration data misaligned with real dynamics, 2) inherent exploration-exploitation tensions in SAC that amplified action variance, and 3) absence of negative samples in offline RL.

These findings highlight key considerations for real-world RL:

1. **Offline data fidelity** is paramount—kinesthetic teaching or teleoperation with precise action recording should replace manual guidance.
2. **Safety mechanisms** must be integrated directly into the learning algorithm (e.g., action clipping, constrained RL).
3. **Careful tuning of the exploration-exploitation balance** is essential—adaptive entropy regularization or conservative policy updates can help prevent unsafe action variance during training.

Looking ahead, future work will focus on further tuning critical hyperparameters—such as temperature and safety constraints—and exploring additional techniques to enhance safety and enable successful real-world training. Besides, introducing more advanced offline RL algorithms can better leverage demonstration data lacking negative examples, thereby reducing value estimation bias and

improving policy robustness. Finally, incorporating model-based RL approaches may further boost sample efficiency.

References

[1] Philip J. Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data, 2023.

[2] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018.

[3] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2024.

[4] Yihang Hu, Pingyue Sheng, Shengjie Wang, and Yang Gao. Mastering agile tasks with limited trials, 2025.

[5] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning, 2021.

[6] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning, 2020.

[7] Jianlan Luo, Zheyuan Hu, Charles Xu, You Liang Tan, Jacob Berg, Archit Sharma, Stefan Schaal, Chelsea Finn, Abhishek Gupta, and Sergey Levine. Serl: A software suite for sample-efficient robotic reinforcement learning, 2025.

[8] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, December 2020.

[9] Homer Walke, Jonathan Yang, Albert Yu, Aviral Kumar, Jedrzej Orbik, Avi Singh, and Sergey Levine. Don't start from scratch: Leveraging prior data to automate robotic reinforcement learning, 2022.

[10] Philipp Wu, Alejandro Escontrela, Danijar Hafner, Ken Goldberg, and Pieter Abbeel. Daydreamer: World models for physical robot learning, 2022.

[11] Tianhe Yu, Garrett Thomas, Lantao Yu, Stefano Ermon, James Zou, Sergey Levine, Chelsea Finn, and Tengyu Ma. Mopo: Model-based offline policy optimization, 2020.